

# Использование безопасных протоколов

## Протоколы прикладного уровня

### Описание проблемы

Небезопасные протоколы, такие как FTP, Telnet, HTTP были разработаны в ранние годы интернета, когда вопросы безопасности не стояли так остро. Эти протоколы не обеспечивают шифрование передаваемых данных, что делает их уязвимыми для перехвата и атак злоумышленников. В результате, они не соответствуют современным стандартам безопасности и не рекомендуются для использования в случаях, где требуется защита данных.

Основные проблемы использования небезопасных протоколов:

1. **Передача данных в открытом виде:** Протоколы типа FTP, Telnet и HTTP передают данные, включая учетные данные (имена пользователей и пароли), в незашифрованном виде. Это делает их уязвимыми для перехвата злоумышленниками.
2. **Атаки типа "человек посередине" (MITM):** Злоумышленники могут перехватывать и изменять передаваемые данные, поскольку трафик не зашифрован.
3. **Уязвимости и эксплойты:** Устаревшие протоколы имеют известные уязвимости, которые легко эксплуатируются.
4. **Отсутствие обновлений:** Многие из этих протоколов больше не поддерживаются, что означает отсутствие обновлений и исправлений для новых уязвимостей.

### Последствия эксплуатации

Какие могут быть последствия при использовании небезопасных протоколов:

1. **Утечка конфиденциальной информации:** перехваченные учетные данные и передаваемые данные могут быть использованы для несанкционированного доступа к системе и информации в ней.
2. **Возможность дальнейших атак:** Компрометация одной части системы может предоставить злоумышленникам возможность проникнуть глубже в инфраструктуру организации, что увеличивает риск дальнейших атак.

### Что делать?

Использовать безопасные протоколы:

- **Использование SFTP или FTPS вместо FTP:** SFTP (SSH File Transfer Protocol) и FTPS (FTP Secure) обеспечивают шифрование данных и защищают их от перехвата.
- **Использование SSH вместо Telnet:** SSH (Secure Shell) обеспечивает безопасный доступ к удаленным системам благодаря шифрованию и аутентификации.



Использование SSH-ключей для аутентификации при подключении к удаленным серверам значительно повышает безопасность по сравнению с использованием логина и пароля. SSH-ключи являются криптографически сильными и практически неуязвимыми для атак методом перебора (брутфорс), тогда как пароли, даже сложные, могут быть скомпрометированы.

**После настройки SSH-ключей рекомендуется отключить аутентификацию по паролю в конфигурации SSH-сервера (/etc/ssh/sshd\_config) для повышения безопасности.**

Убедитесь, что приватный ключ защищен паролем и хранится в безопасном месте.

- **Использование HTTPS вместо HTTP:** HTTPS (HyperText Transfer Protocol Secure) обеспечивает шифрование данных с использованием TLS (Transport Layer Security), что защищает передаваемую информацию от перехвата и подмены.

## SSL/TLS

Secure Socket Layer (SSL) – это оригинальный протокол, который использовался для обеспечения шифрования HTTP-трафика в форме HTTPS. Существовало две публично выпущенные версии SSL – версии 2 и 3. Обе они имеют серьезные криптографические недостатки и больше не должны использоваться.

По разным причинам следующая версия протокола (фактически SSL 3.1) была названа Transport Layer Security (TLS) версии 1.0. Впоследствии были выпущены версии TLS 1.1, 1.2 и 1.3.



Термины SSL, SSL/TLS и TLS часто используются как взаимозаменяемые, и во многих случаях SSL используется для обозначения более современного протокола – TLS.

## Описание проблемы

**Небезопасные и устаревшие версии TLS**, такие как TLS 1.0 и TLS 1.1, были разработаны в начале 2000-х годов и содержат множество уязвимостей, которые могут быть использованы злоумышленниками для атаки на системы безопасности. Использование этих версий не соответствует современным стандартам безопасности и может подвергнуть данные и систему значительному риску.

Некоторые из уязвимостей, присутствующих в устаревших версиях TLS:

1. **POODLE (Padding Oracle On Downgraded Legacy Encryption)**: Уязвимость в TLS 1.0 и 1.1, позволяющая злоумышленнику расшифровать трафик, используя атакующую манипуляции с padding.
2. **BEAST (Browser Exploit Against SSL/TLS)**: Уязвимость, позволяющая атакующему перехватывать и расшифровывать HTTPS-сессии, использующие TLS 1.0.
3. **CRIME (Compression Ratio Info-leak Made Easy)**: Уязвимость, использующая компрессию SSL/TLS для утечки данных. При восстановлении содержимого файлов cookie она позволяет злоумышленнику осуществить перехват сеанса аутентифицированной веб-сессии, что дает возможность для проведения дальнейших атак.

## Что делать?

### Используйте только надежные протоколы

Веб-приложения общего назначения должны по умолчанию использовать TLS 1.3 (при необходимости поддерживать TLS 1.2) с отключением всех остальных протоколов.

### Используйте только стойкие алгоритмы шифрования

Существует большое количество различных шифров (или наборов шифров), поддерживаемых TLS, которые обеспечивают различные уровни безопасности. По возможности следует включать только шифры **GCM**. Однако если необходимо поддерживать устаревшие клиенты, то могут потребоваться другие шифры. Как минимум, следующие типы шифров должны быть всегда отключены:

- Null
- Anonymous
- EXPORT

### Отключите сжатие

Сжатие TLS должно быть отключено для защиты от уязвимости CRIME, о которой было рассказано выше.

### Не отключать проверку SSL-сертификатов

Игнорирование ошибок SSL-сертификатов и отключение проверки имени серверного сертификата создает значительные риски безопасности. Это действие позволяет злоумышленникам легко провести атаку типа «человек посередине» (MITM), так как клиент не проверяет подлинность сервера. В результате передаваемые данные могут быть перехвачены или изменены, что может привести к утечке конфиденциальной информации и компрометации системы. Отключение таких проверок недопустимо с точки зрения безопасности.



### Протестируйте конфигурацию сервера

Существуют онлайн-инструменты, которые можно использовать для быстрой проверки конфигурации сервера, например утилита от SSL Labs:

<https://www.ssllabs.com/ssltest>



Mozilla Foundation предлагает простой в использовании [генератор безопасных конфигураций](#) для веб-серверов, баз данных и почтовых серверов. Этот инструмент позволяет администраторам сайтов выбрать используемое ими программное обеспечение и получить файл конфигурации, оптимизированный с точки зрения безопасности и совместимости для широкого спектра версий браузеров и серверного программного обеспечения.

